SAP Security Expert Curriculum

Introduction to Cryptography

Session 3: Asymmetric Encryption

Overall Agenda for Introduction to Cryptography

Session 1: Introduction and Crypto Basics

Session 2: Symmetric Encryption

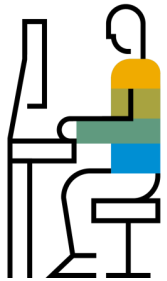
Session 3: Asymmetric Encryption

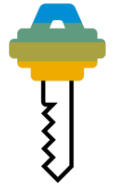
Motivation

Motivation

- Alice has a plain text message for Bob.
- Bob has created a strong symmetric key.
- But the channel is public and not trustworthy.
- How to share a symmetric key safely?

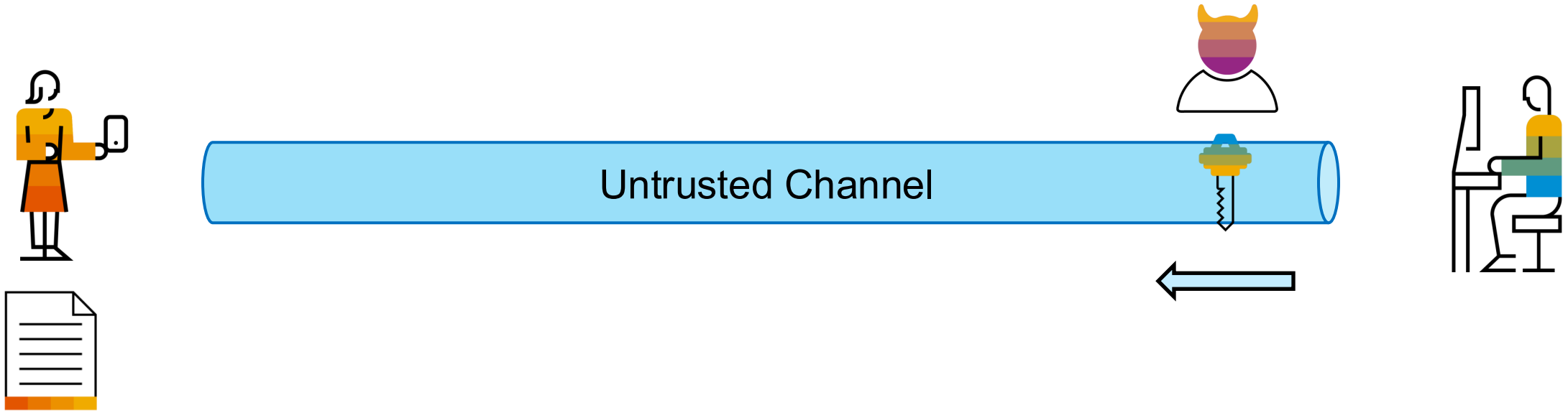
Untrusted Channel





Motivation

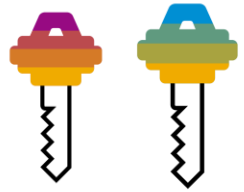
- Attacker can easily eavesdrop on key.
- Attacker will be able to read all future communication using that key.
- Attacker will be able to read all previous communication using that key.



Motivation

- What if each party has 2 keys?
  - one private to be kept secret
  - one public to be shared with everyone
- Keys form a related keypair
  - Mathmatical dependent
  - Created together

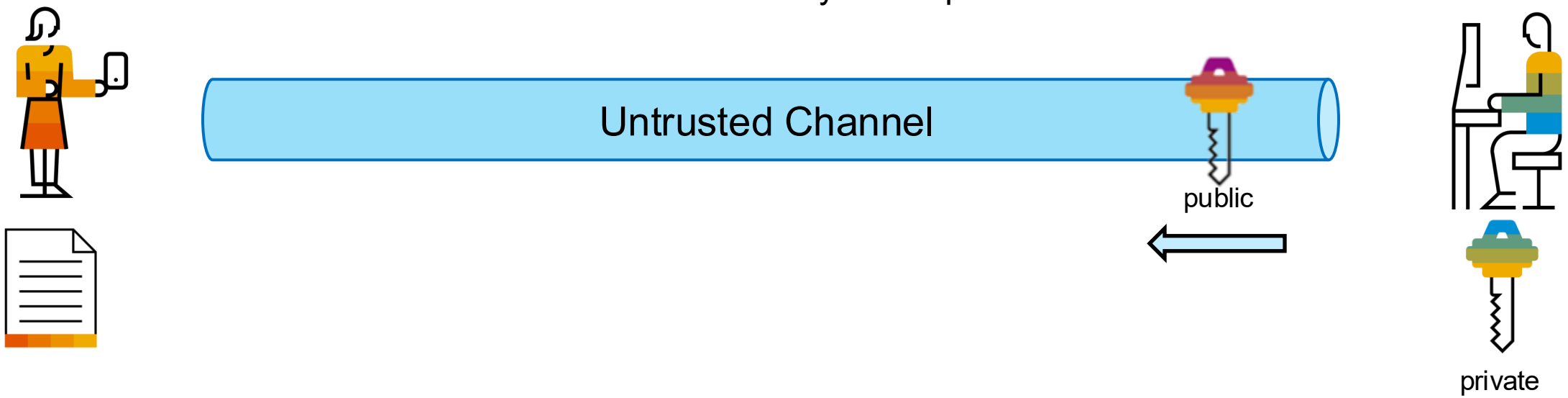
Untrusted Channel



public private

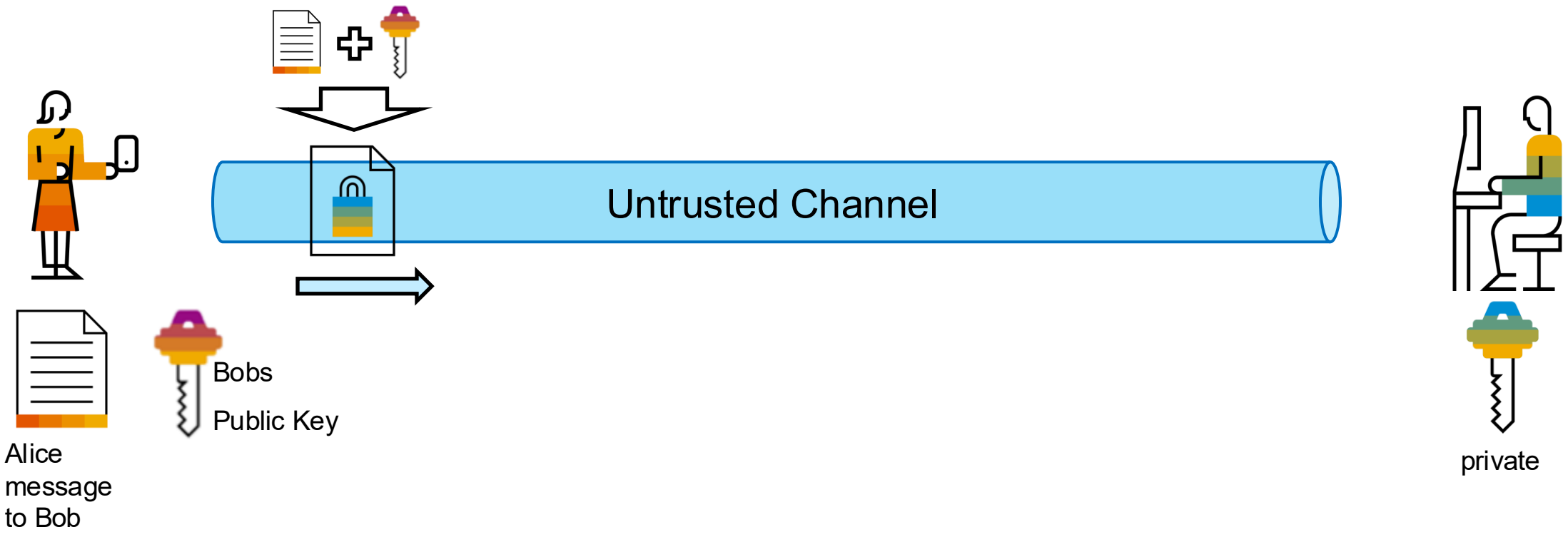
Motivation

- Idea: Public Key
  - Can be given to ANYONE
  - Can be uploaded into public registries
- Idea: Private Key
  - Must be strictly secret
  - If leaked security is compromised



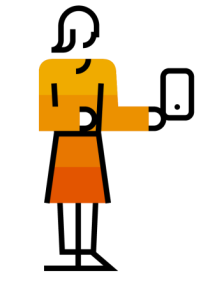
Motivation

- Alice uses Bobs public key to encrypt a message she wants to send to him



Motivation

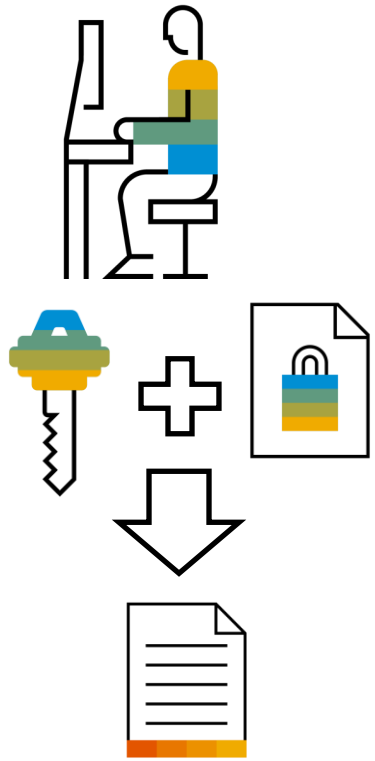
- Bob uses his private key to decrypt the ciphertext
- No need to exchange a symmetric key beforehand



Alice message to Bob

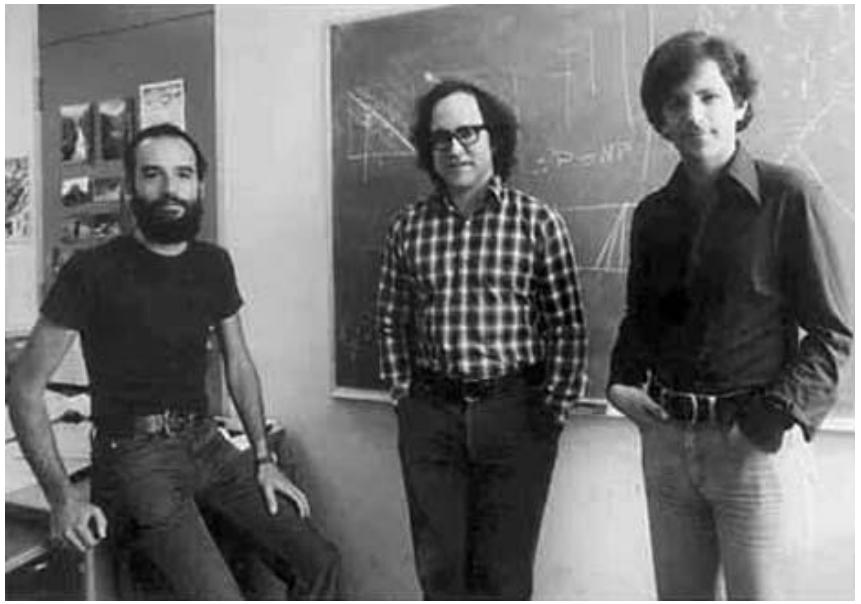
Bobs Public Key

Untrusted Channel



RSA

Rivest, Shamir, Adleman



http://www-history.mcs.st-and.ac.uk/~history/PictDisplay/Adleman.html

RSA – What is the trapdoor? Large Integer factorization

Let N be an integer $N = p \cdot q$ with primes p, q.

p, q are secret - N is public.

It is just a multiplication but if you only know N it is very hard to find p and q.

p, q are 3000 bits large (this is currently considered secure)

p=
7940368955088604599005728506324482928247924602890336875830081585874113623860050696642634922346458717313411320034029729008119587402079193175625188541869225967852139618744962141431135557127874949673121812042387478219017428980796404634758332412880411051414209370694500846632646000295548787232201297900286856555488074959997228772407411442289783319194728224270862652250623195395028814481024507605296869894569541287714903631818790164788933408592148804381144577168682406537 (466 digits)

q=
8687917680002890462904260428866157096201433783528886236998272466402922071838655890842019476834145199400996000226007581135209849864165403733499966582252694597458823761956793442739295221407031167929848047420413879558946163104211842669136531484387333884884845502649728115577189981955866827024818923453560602192336373502305396529603350189033165807585898606925777244504178877061866213063421079332274471386869511191315895908211735649026604137224921788110992020111984081463 (466 digits)

p*q = N =
6898527183066036520414200386628685963474041043531831670890588221852488396481200146275643122992937315361467336959657906769692564865673032487345775739465604884780122598572484207392458346085537380777721809613539730756828458652009014231755888393129933885579994602679514578519955774063710540885362025040392070984610901684675040370545600287848396845366962847810312594719872374337306877888078441660100091602640319663278669934785292499823260106387436352820186033047221134824122500215455282858961117420912470148587330321865722530867222149743328747487917596855822990581797928546493757713380798483760977238982645327836447882446124874989433587221505887893417721513831839572652345713845715155585551774483277712943082169610383625274670722860757397735030028489629295565629456818480657546465670232316146962386510861293426356279218211988000194381319831017980137638130461835823455260504127651374169414322057311679761043547901482684125524601429172363 1 (932 digits)

Factorization Assumptions

Let N be an integer $N = p \cdot q$ with primes p, q.

Cryptographers believe that it is hard to extract p or q given only N.

(For large numbers. N has at least 900 digits. Known efficient algorithm – Shor-Algorithm but only works on ideal Quantum computer that is not yet available)

Implications:

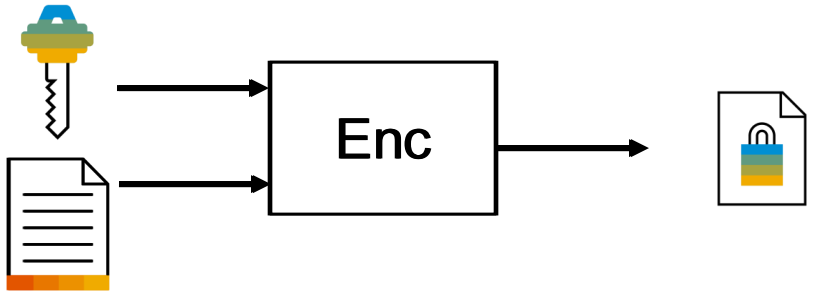
- $\phi(N) = (p-1) \cdot (q-1)$ is easy to compute given p or q
- But it is hard to compute $\phi(N)$ given only N. (You would need to factorize N first to get p and q)

RSA Cryptosystem: Key Generation

1. Pick two random prime numbers (at least 1024 bits each)
2. Calculate N = p * q and $\phi(N)$ = (p-1) * (q-1)
3. Pick e such that:
   - 1 < e < $\phi(N)$
   - gcd(e, $\phi(N)$) = 1 (relatively prime if greatest common divisor is 1)
   - often fixed to 65537 which is a good pick (all primes are relatively prime to everything else ☺)
4. Calculate d such that e * d mod $\phi(N)$ = 1
   - Reason why we need $\phi(N)$
     - N is public and calculating *e * d mod N = 1* would be easy
     - Without factorizing N -> no p and q -> no $\phi(N)$ -> no inverse to e mod $\phi(N)$

Public key: (e, N) Private key: (d, N)

RSA Cryptosystem: Encryption

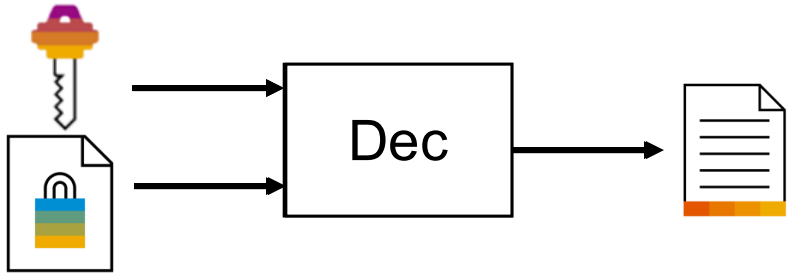


Encryption with public key (e, N) and plaintext $M \in \mathbb{Z}_N$:

$$M^e \bmod N = C$$

M = Plaintext
e = public Exponent
N = public Modulus
C = Ciphertext

RSA Cryptosystem: Decryption



Decryption with private key (d, N) and ciphertext C:

$$C^d \bmod N = M$$

C = Ciphertext
d = private Exponent
N = public Modulus
M = Plaintext

- $C^d \bmod N = (M^e)^d \bmod N = M^{e \cdot d} \bmod N = M^{e \cdot d \bmod \phi(N)} \bmod N = M^1 = M$
  - Remember that $e \cdot d \bmod \phi(N) = 1$

Exercise – FOR EDUCATIONAL PURPOSE ONLY

Implement a small example for RSA with the following parameters:

- P = 23, Q = 31
  - What is N?
  - What is $\phi(N)$?
- e = 127
  - What is the public key?
  - What is the corresponding private key? (find e.g. via brute force)

- Encrypt Message $m = 41$ with this secret key and decrypt the resulting ciphertext afterwards, that is, evaluate the correctness of RSA.

Open Problems:

- Deterministic Encryption
- Plaintext must be in $\mathbb{Z}_N$

General Attacks Against RSA

Try to factor N

Side channel attacks *(example: Timing-Attack against Square-&-Multiply)*

→ Do not implement RSA algorithm – use approved libraries

Using leaked $\phi(N)$

→ Vieta (simple quadratic equation), so keep $\phi(N)$ secret, too!

Finding RSA modulus with a common prime factor

→ Lenstra et al. 2012: https://eprint.iacr.org/2012/064.pdf
„two out of every one thousand RSA modulus that we collected offer no security“

Insider attacks when company uses a common modulus N for all employees

→ Do not reuse modulus N, always generate new p and q

Attacks Against Plain RSA

- No semantic security
  - try out messages and other attacks for deterministic encryption
- Size Constraints message too short:
  - M=3, public Key (e=13, N= 3.000.009)
  - Encryption: 3^{13} mod 3.000.009 = 1.594.323 mod 3.000.009 = 1.594.323 = 3^{13} ????
  - If plaintext too short -> $\sqrt[e]{C} = M$; $\sqrt[13]{1.594.323} = 3$
  - we need some padding so modulus has effect

Never use plain RSA without appropriate all-or-nothing transformation, like OAEP

Attacks Against Plain RSA

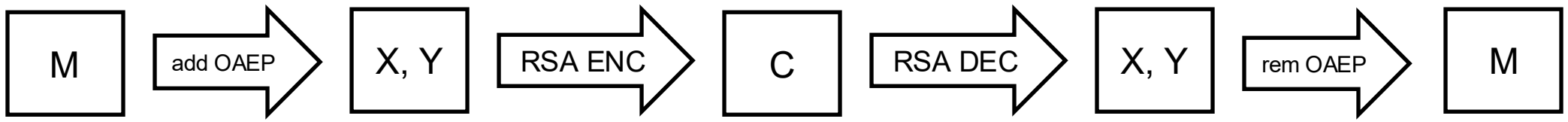
Same message sent to multiple recipients

→ can be recovered using *Chinese Remainder Theorem* (some math known already in the 3rd century…)

Never use plain RSA without appropriate all-or-nothing transformation, like OAEP

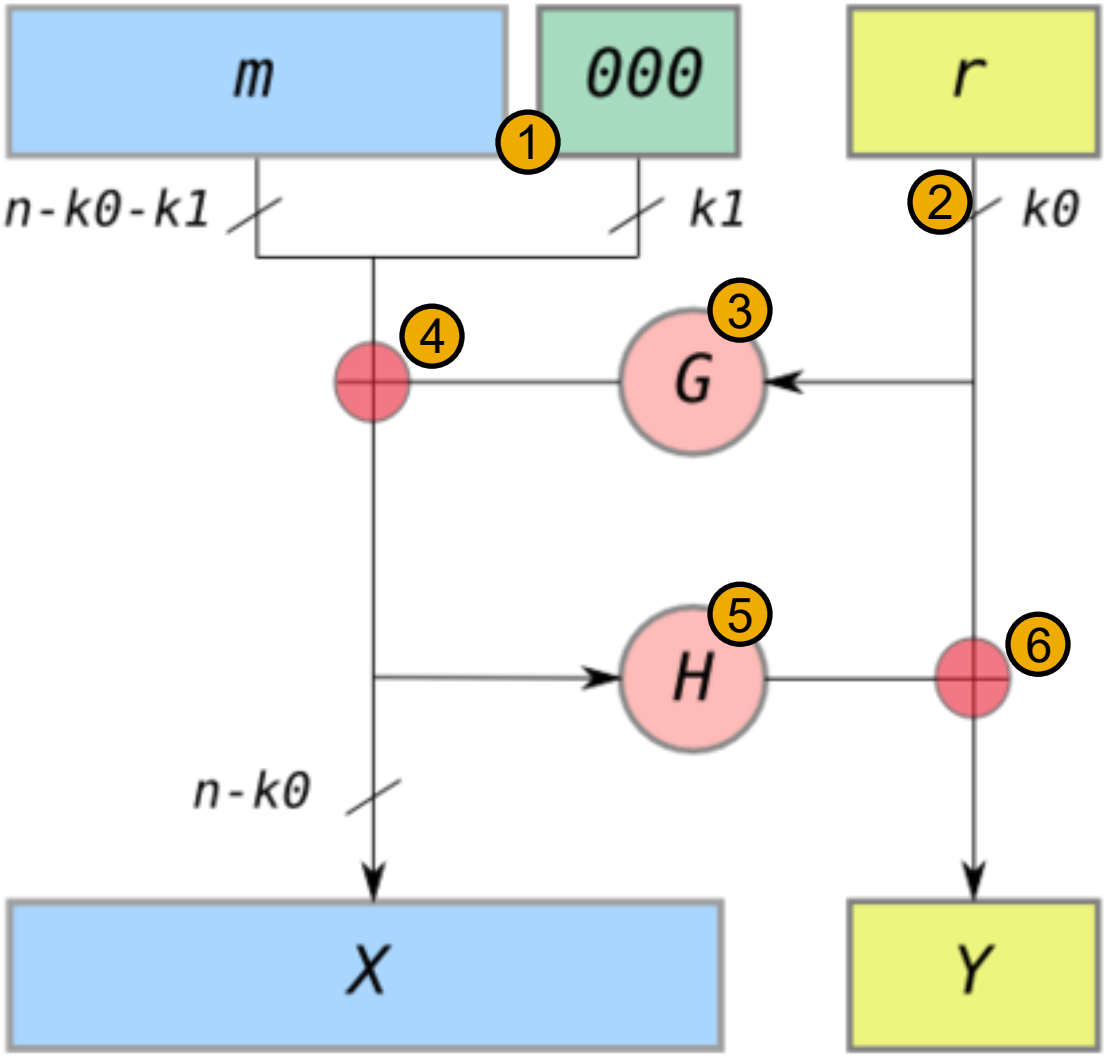
OAEP - Optimal Asymmetric Encryption Padding

- Let us fix the problems of RSA
  - Make sure the message is sufficiently long so modulus N has an effect -> add padding to plaintext
  - Make sure there are no semantic features that an attacker could analyze -> add randomness
  - Make it easy to add and remove this all-or-nothing transformation before encryption and after decryption



OAEP - Optimal Asymmetric Encryption Padding

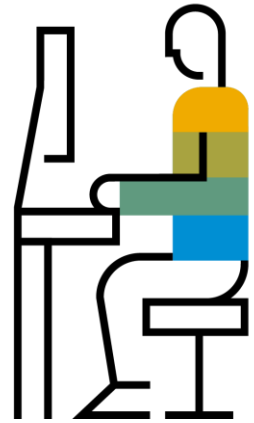
1. Pad message m with k_1 „0“
2. Choose a random r with k_0 Bit
3. Compute hash G of r
4. X = m00...0 ⊕ G(r) (XOR)
5. Compute hash H of X
6. Y = r ⊕ H(X) (XOR)



Hybrid Encryption

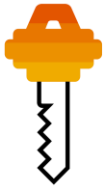
Combine Asymmetric Encryption and Symmetric Encryption.

- Use Asymmetric Encryption for Key Distribution (of a symmetric key).
- Use that Symmetric Key for actual Payload Encryption.



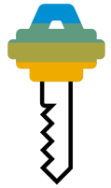
Alice
Message
to Bob

Symmetric
Key



Bobs
Public Key

Bobs
Public Key

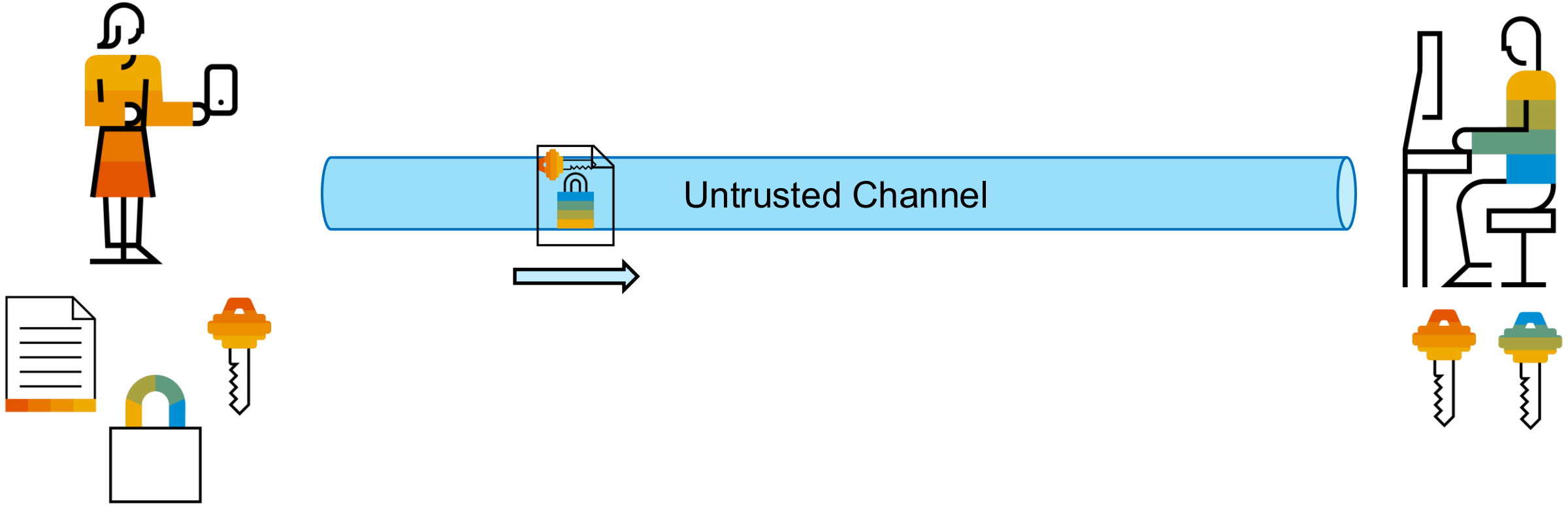


Bobs
Private Key

Hybrid Encryption

Combine Asymmetric Encryption and Symmetric Encryption.

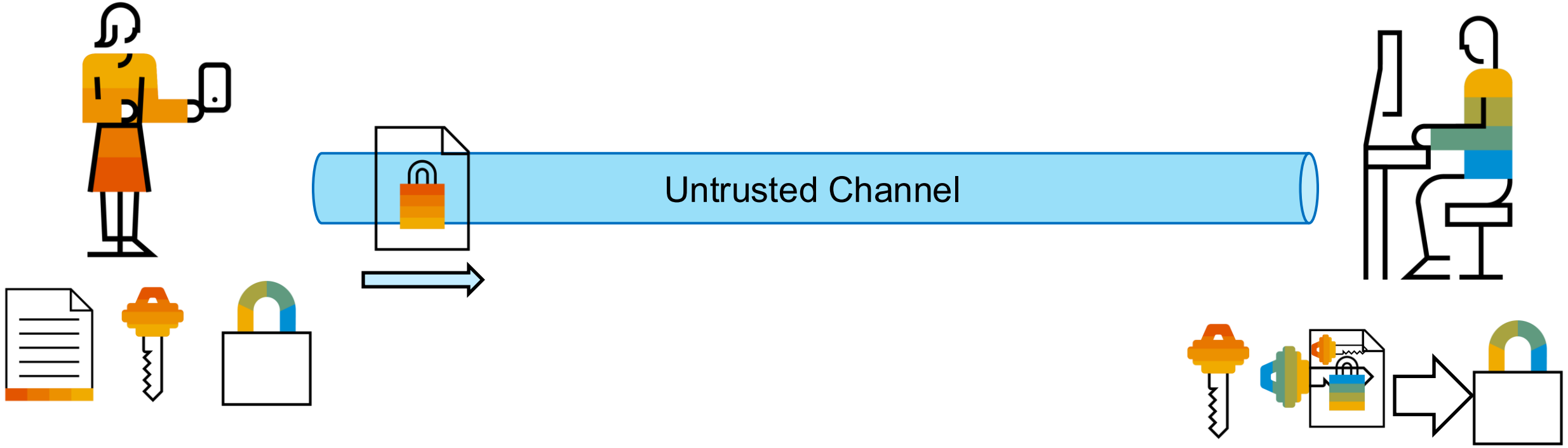
- Use Asymmetric Encryption for Key Distribution (of a symmetric key).
- Use that Symmetric Key for actual Payload Encryption



Hybrid Encryption

Combine Asymmetric Encryption and Symmetric Encryption.

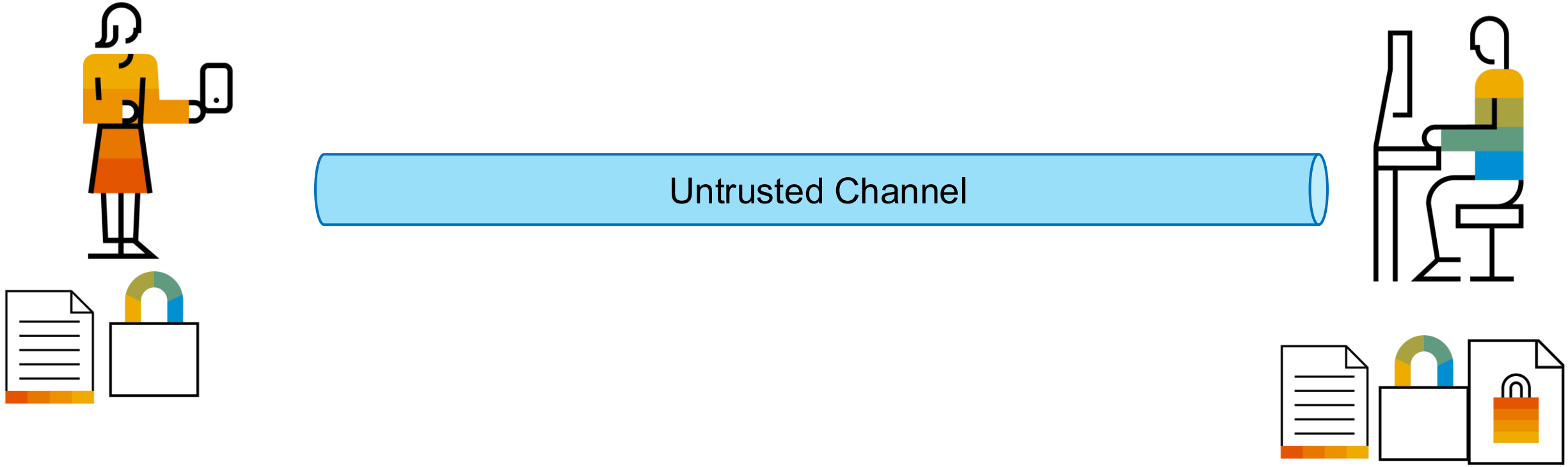
- Use Asymmetric Encryption for Key Distribution (of a symmetric key).
- Use that Symmetric Key for actual Payload Encryption.



Hybrid Encryption

Combine Asymmetric Encryption and Symmetric Encryption.

- Use Asymmetric Encryption for Key Distribution (of a symmetric key).
- Use that Symmetric Key for actual Payload Encryption.



Signatures

Digital Signatures

creating an e-mail signature
(or any other document or file)

e-mail sender side

e-mail document

hashing function

SHA-512

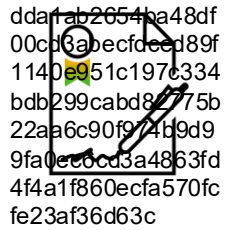
hash value

175f859121ffd28c9
3688f143652e9878
5b994e8db9e0afc5
5c4e3c7d3500bee6
b2631fa47cfd9f123
3d5712f0c41fb2bcd
12f4b3ad5ddb5f65c
7b87052bd1c2

signing with secret private key (d, N) in RSA

$hash^d \bmod N$

signature



- can vary in size
- signing very large documents can lead to problems in rsa (size constraints)

- always fixed size of 512 bits

- signature can be attached to document as proof for authenticity and integrity
- only holder of secret key can create valid signatures
- if document was modified it won‘t match the signature

Digital Signatures

validating an e-mail signature
(or any other document or file)

e-mail recipient side

e-mail document

hashing function

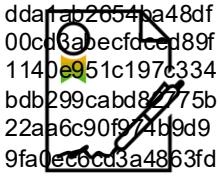
SHA-512

hash value

175f859121ffd28c9
3688f143652e9878
5b994e8db9e0afc5
5c4e3c7d3500bee6
b2631fa47cfd9f123
3d5712f0c41fb2bcd
12f4b3ad5ddb5f65c
7b87052bd1c2

compare
hashes

signature



ddaffab2654ba48df
00cd3abecfdced89f
1140e951c197c334
bdb299cabd8e775b
22aa6c90f974b9d9
9fa0ec6cd3a4863fd
4f4a1f860ecfa570fc
fe23af36d63c

verifying with public key
(e, N) in RSA

signature^{e} mod N

hash value

175f859121ffd28c9
3688f143652e9878
5b994e8db9e0afc5
5c4e3c7d3500bee6
b2631fa47cfd9f123
3d5712f0c41fb2bcd
12f4b3ad5ddb5f65c
7b87052bd1c2

If the hashes match we know:

- e-mail was signed with the private key belonging to the public key proving authenticity of the sender
- E-Mail or document was not modified on transfer
- (as long as the private key is really secret)

Digital Signatures

Encryption/Decryption vs
Signing/Verifying

- while mathmatical operations for Decryption/Signing and Encryption/Verification are the same they should not be confused

- signing/decryption may mathmatically be the same but:
  - signing operation is not applied on ciphertext
  - output of signing operation is not plain text
  - logically signing != decryption

RSA: Key Generation (we have seen this before)

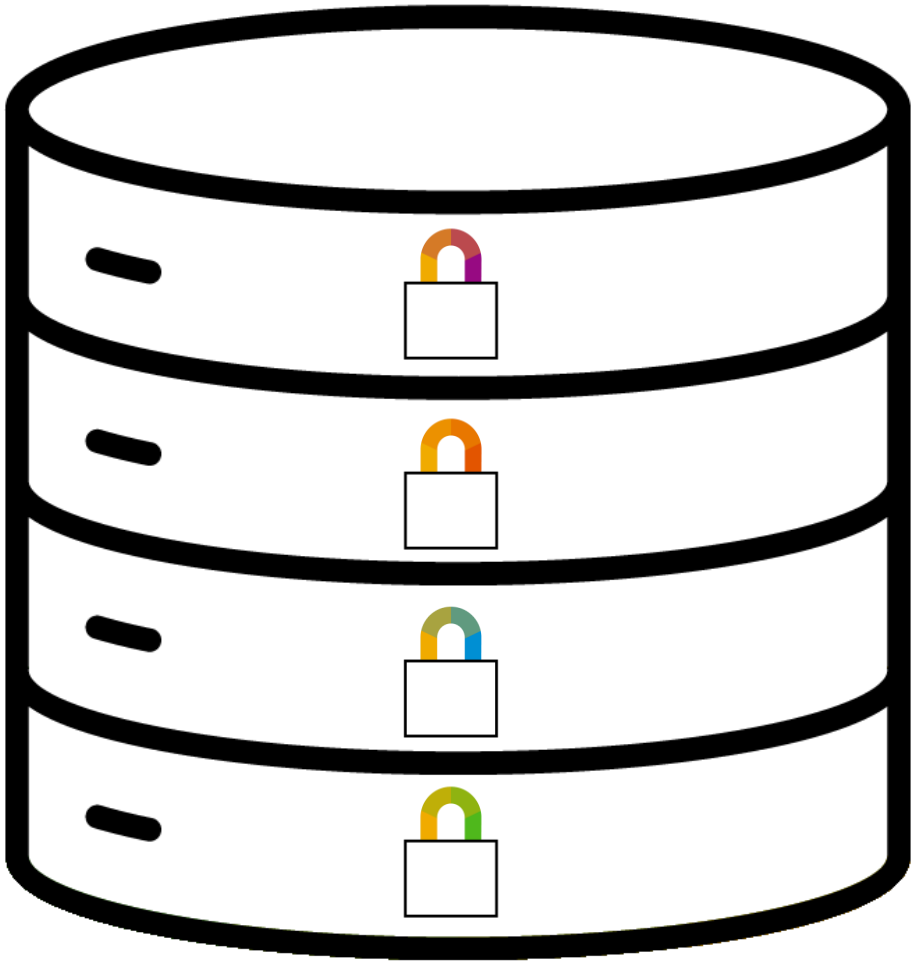
1. Pick two random prime numbers (at least 1024 bits each)
2. Calculate N = p * q and $\phi(N)$ = (p-1) * (q-1)
3. Pick e such that:
   - 1 < e < $\phi(N)$
   - gcd(e, $\phi(N)$) = 1 (relatively prime if greatest common divisor is 1)
   - often fixed to 65537 which is a good pick (all primes are relatively prime to everything else ☺)
4. Calculate d such that e * d mod $\phi(N)$ = 1
   - Reason why we need $\phi(N)$
     - N is public and calculating *e * d mod N = 1* would be easy
     - Without factorizing N -> no p and q -> no $\phi(N)$ -> no inverse to e mod $\phi(N)$ ->

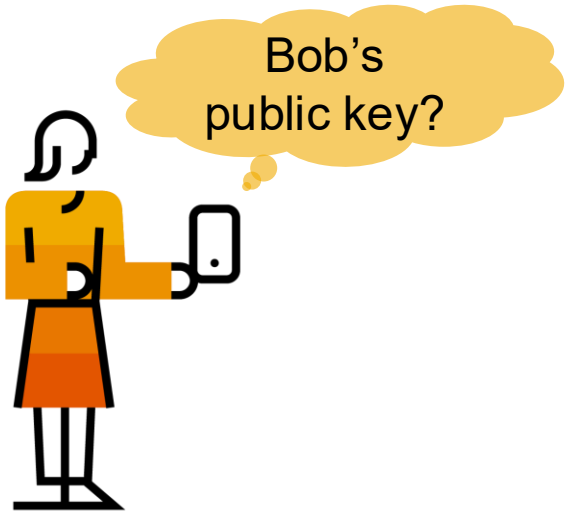
Public key: (e, N) Private key: (d, N)

Perfect Forward Secrecy
and
Diffie-Hellman Key Exchange

Perfect Forward Secrecy

Assume a central long term public key register

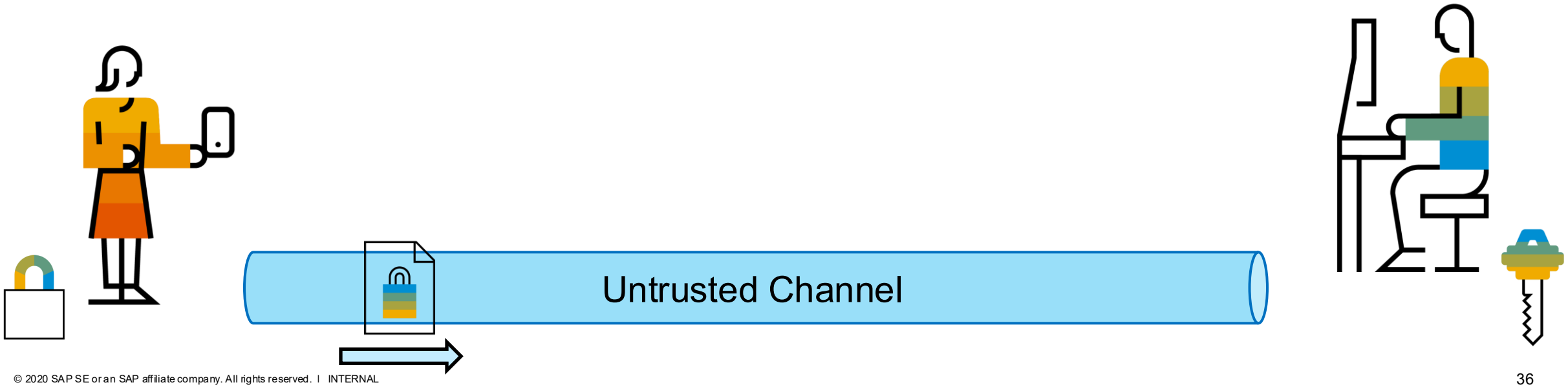




Perfect Forward Secrecy

Assume a central long term public key register

All messages are transferred encrypted (hybrid encryption)



Perfect Forward Secrecy

Assume a central long term public key register

All messages are transferred encrypted (hybrid encryption)

An attacker keeps all encrypted messages…

Perfect Forward Secrecy

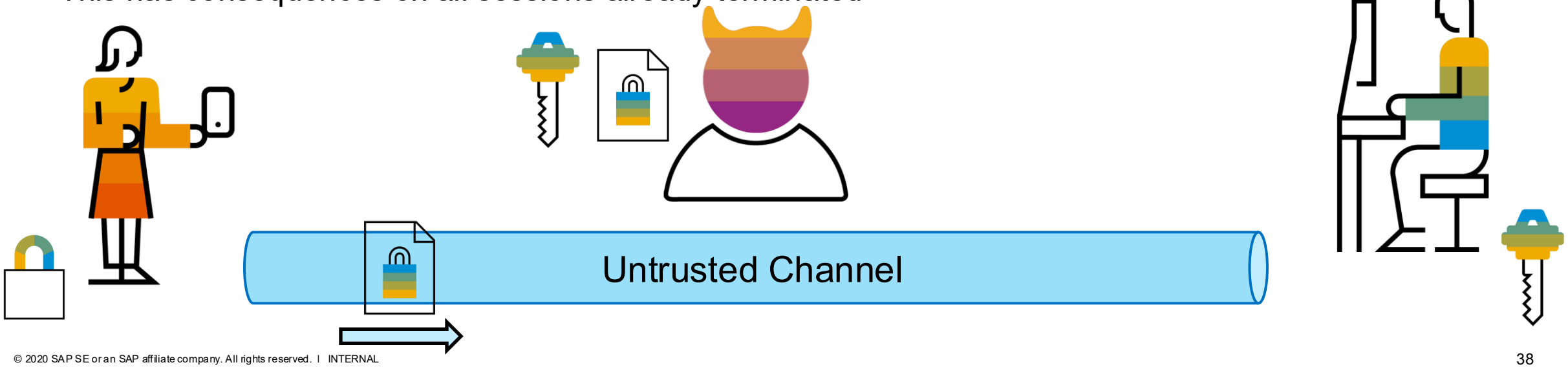
Assume a central long term public key register

All messages are transferred encrypted (hybrid encryption)

An attacker keeps all encrypted messages…

… until the secret key gets compromised

- This has consequences on all sessions already terminated



Diffie-Hellman Key Exchange – What is the trapdoor? Discrete Logarithm

- $g^x \, mod \, p = a$
- -> this is the easy way g, p and a can be used publicly, x is secret

- The inverse function -> $log_g \, a = x \, (mod \, p)$ – attacker need to figure out the x

- -> known efficient algorithm again: Shor-Algorithm for ideal quantum computers. If quantum computers get better we something else.

- Also p is a 2048 bit prime (current NIST specs)
- x is at least 1024 bits
- g can be chosen should fulfill some criterias (some math here)

- Knowing a, g, p and trying out to find x is very hard

Diffie-Hellman Key Exchange

First public key protocol (initiated entire public key research)

Used to exchange a pairwise symmetric key

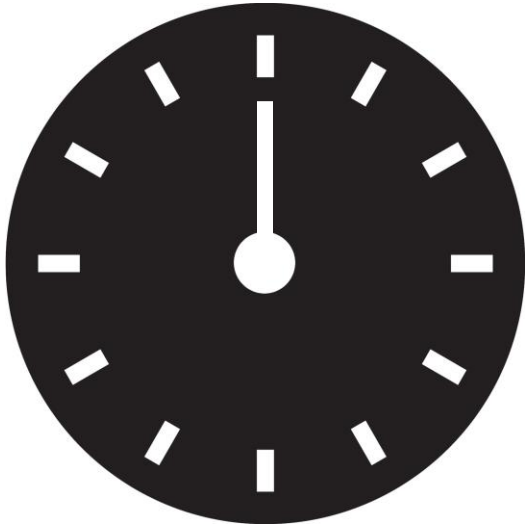
These keys are then used as session keys and discarded after session termination

- Keys for terminated sessions cannot be leaked

Cyclic Groups and Discrete Logarithms

Example for modular arithmetic $\mathbb{Z}_N \cong [0, \ldots, 11]$

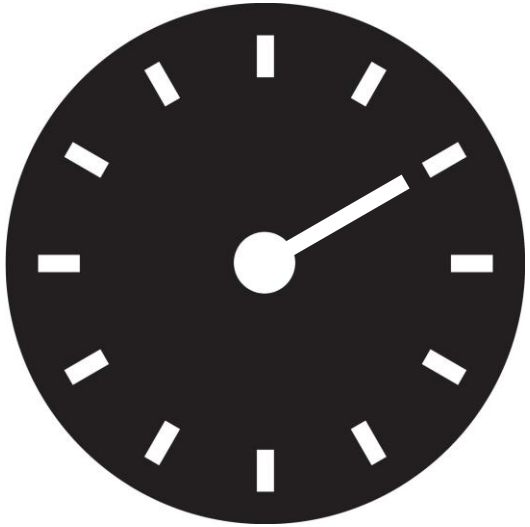
- $0 \cong$ 12 o clock



Cyclic Groups and Discrete Logarithms

Example for modular arithmetic $\mathbb{Z}_N \cong [0, \ldots, 11]$

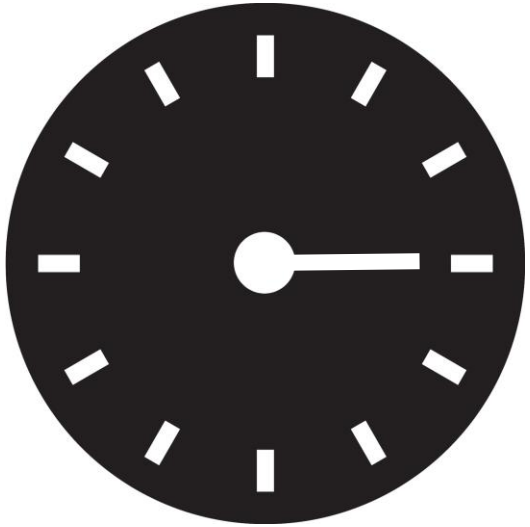
- $0 \cong$ 12 o clock
- $14 \cong$ 2 o clock



Cyclic Groups and Discrete Logarithms

Example for modular arithmetic $\mathbb{Z}_N \cong [0, \ldots, 11]$

- $0 \cong$ 12 o clock
- $14 \cong$ 2 o clock
- ...
- $3^3 = 27 \cong$??



Cyclic Groups and Discrete Logarithms

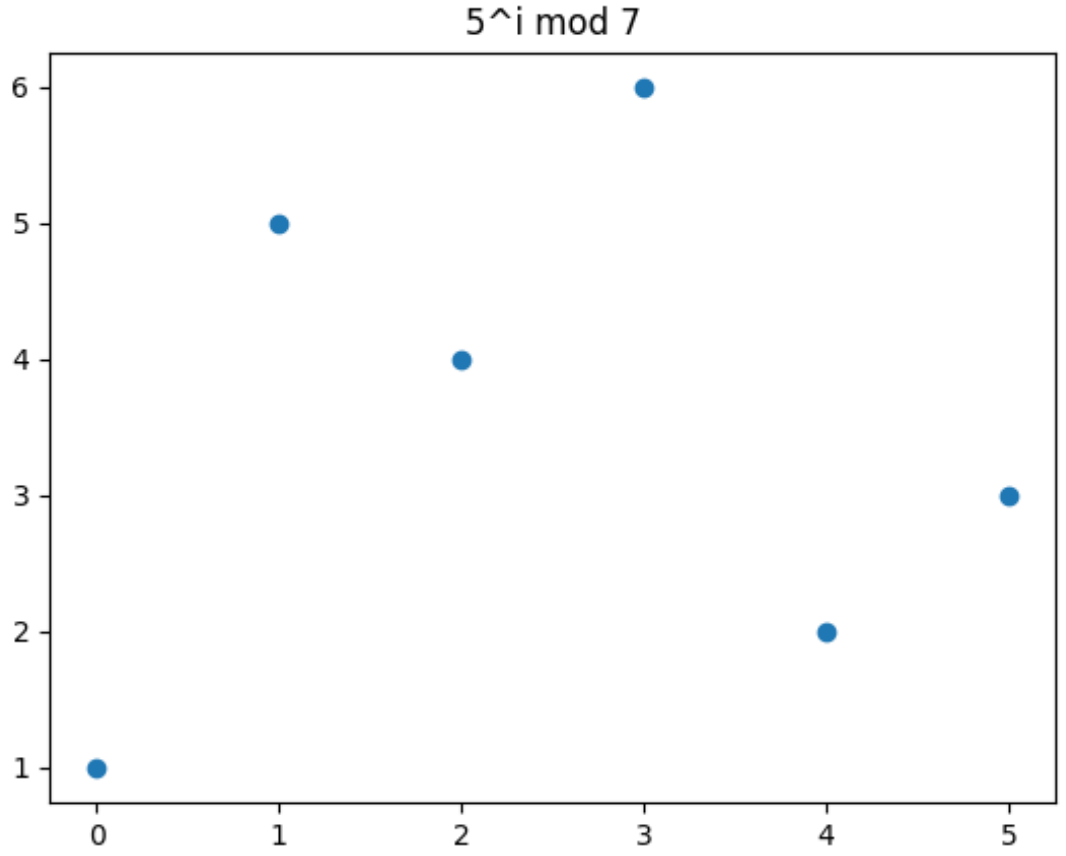
For prime p as modulus one can find an element g such that

- $\langle g \rangle = [(g^i \text{ mod p}) \text{ for } i \text{ in range(0, p-1)}] = [1, \ldots, \text{p-1}]$

Such elements are called generators

Discrete logarithm assumption (p with bit size > 2048)

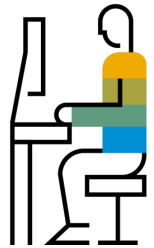
- Given g, x, p it is easy to calculate $g^x \; mod \; p$
- Given g, g^x, p it is hard to calculate x
- There is no known algorithm to efficiently solve the Discrete Log problem



DH-Protokoll

$x \leftarrow [1, p-1]$

g, p



$A = g^x \ mod \ p$

$y \leftarrow [1, p-1]$

$B = g^y \ mod \ p$

$B^x \ mod \ p = \ (g^y)^x mod \ p = g^{yx} mod \ p = g^{xy} mod \ p$

$A^y \ mod \ p = \ (g^x)^y mod \ p = g^{xy} mod \ p$

Shared Secret: $g^{xy} mod \ p$

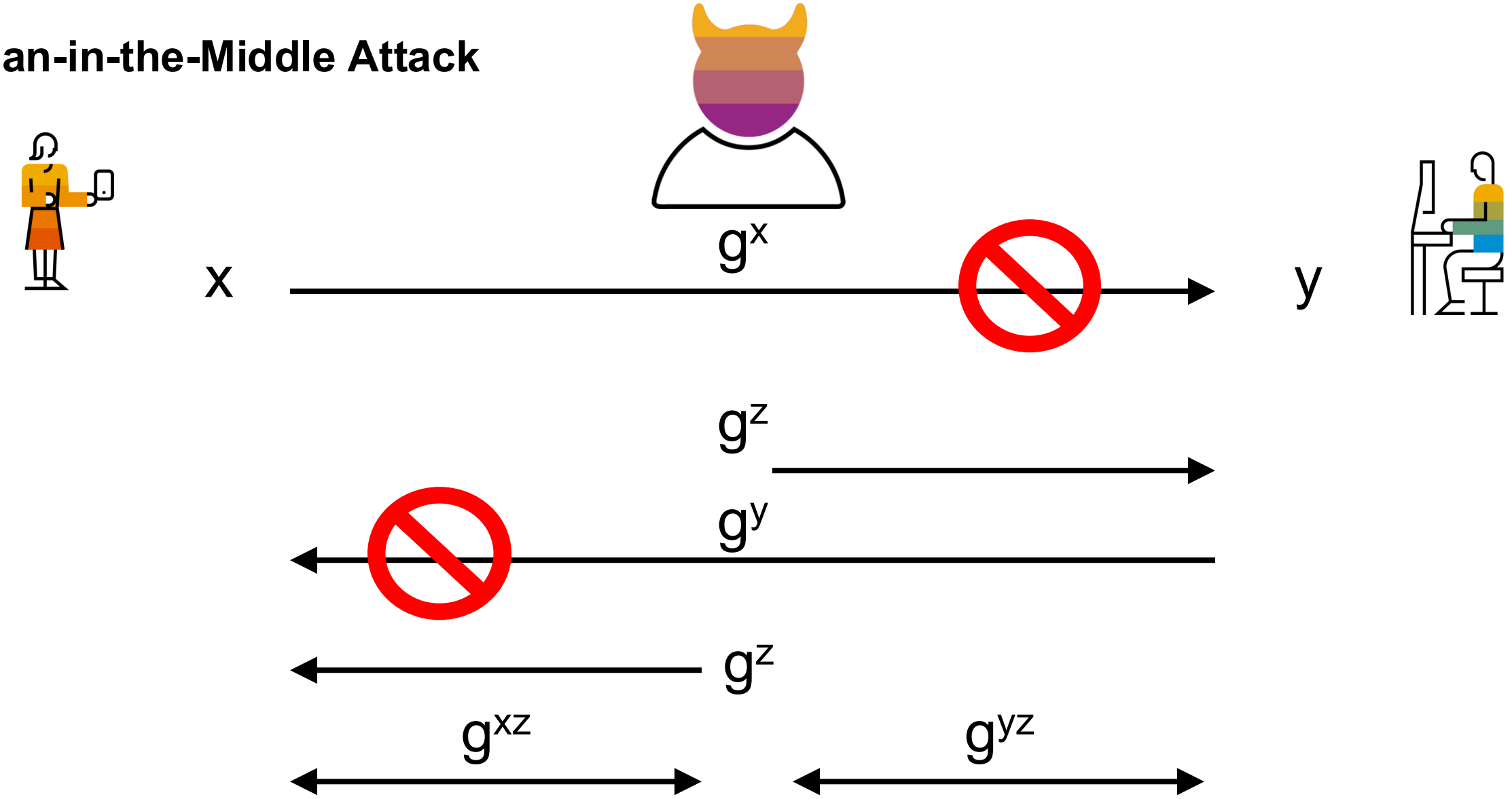
Exercise – FOR EDUCATIONAL PURPOSE ONLY

Implement a small example for Diffie-Hellman key exchange with the following parameters:

- P = 23,
- Generator g = 5
- Alice's secret x = 12
- Bob' secret y = 19

- What is the public value generated by Alice?
- What is the public value generated by Bob?
- What is the shared secret Bob and Alice retrieve?

Man-in-the-Middle Attack



Signatures are crucial!

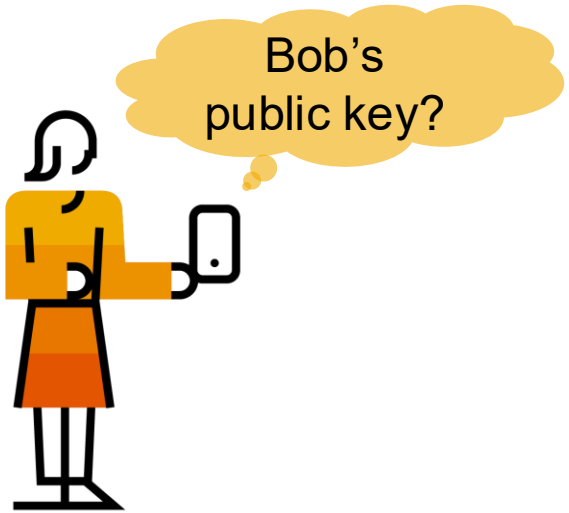
PFS Solution

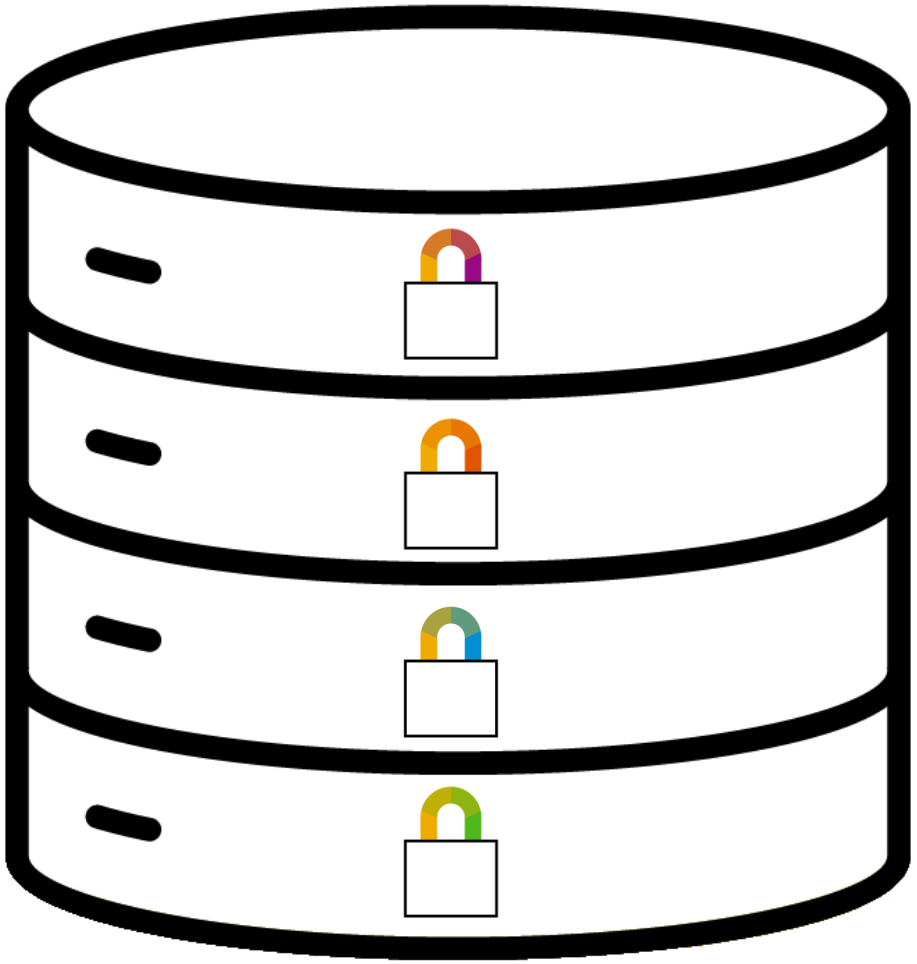
Assume a central long term public key register

Use this public key for digital signatures only

Negotiate session key via signed(!) Diffie Hellman

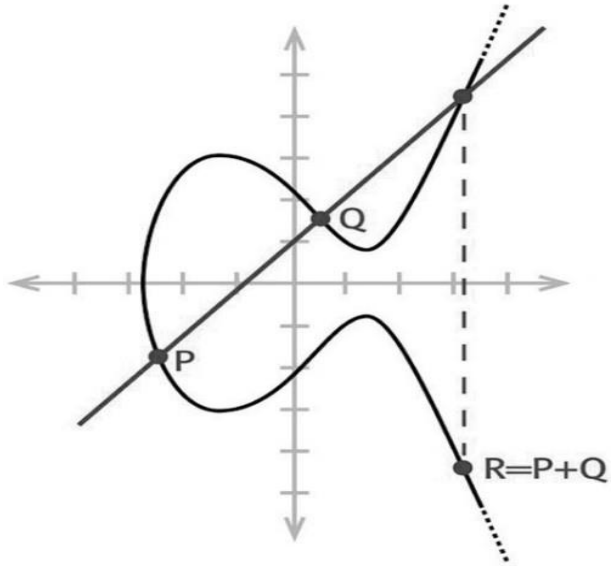
All terminated sessions remain secure even after secret key corruption

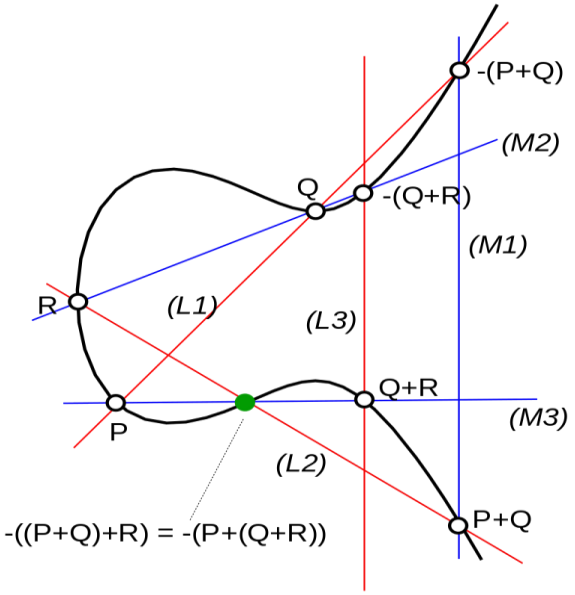




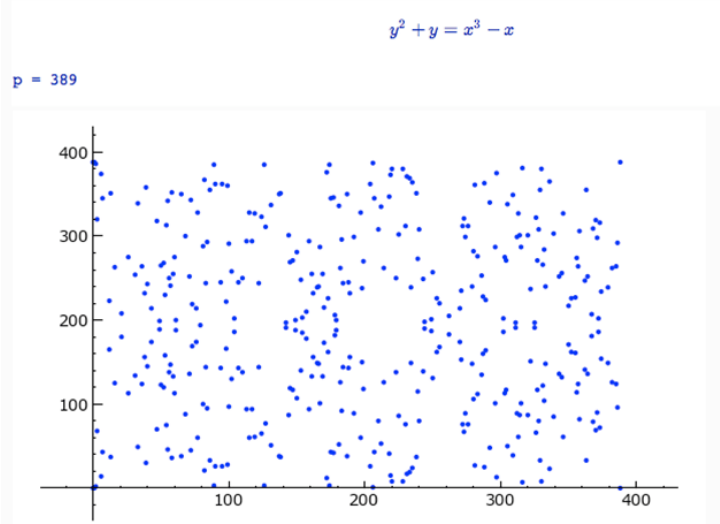
Alternative for picking key pairs P and Q: Elliptic Curves

- $y^2 = x^3 + ax + b$
- Define point addition
- Looks nice over R, but not in finite fields (see below)





http://www.wikiwand.com/de/Satz_von_Cayley-Bacharach



Why care?

- Inversion is less efficient, hence short key lengths can be used and encryption is more efficient
- Important for devices with low computational power
  - RFID, Smart Card, Smart Meter

- EC can be used for digital signatures, random number generators and DH key exchange

Elliptic Curves Cryptography

ECDLP

- Given:
  - EC E: $y^2 = x^3 + ax + b \; mod \; p$ (coefficients a, b and prime p)
  - E cardinality (number of elements): $\#E$
  - generator point G
  - point T
- it is hard to find integer d, where $1 \leq d \geq \#E$ and $d \cdot G = G + G + \ldots + G = T$

Private key is a randomly chosen integer d

Public key is the point $T = d \cdot G$

Keylength Equivalence

Protection	Symmetric	Factoring Modulus	Discrete Logarithm Key	Discrete Logarithm Group	Elliptic Curve	Hash
Legacy standard level *Should not be used in new systems*	80	1024	160	1024	160	160
Near term protection *Security for at least ten years (2018-2028)*	128	3072	256	3072	256	256
Long-term protection *Security for thirty to fifty years (2018-2068)*	256	15360	512	15360	512	512

Recommendation by ECRYPT-CSA (Coordination & Support Action)

https://www.keylength.com